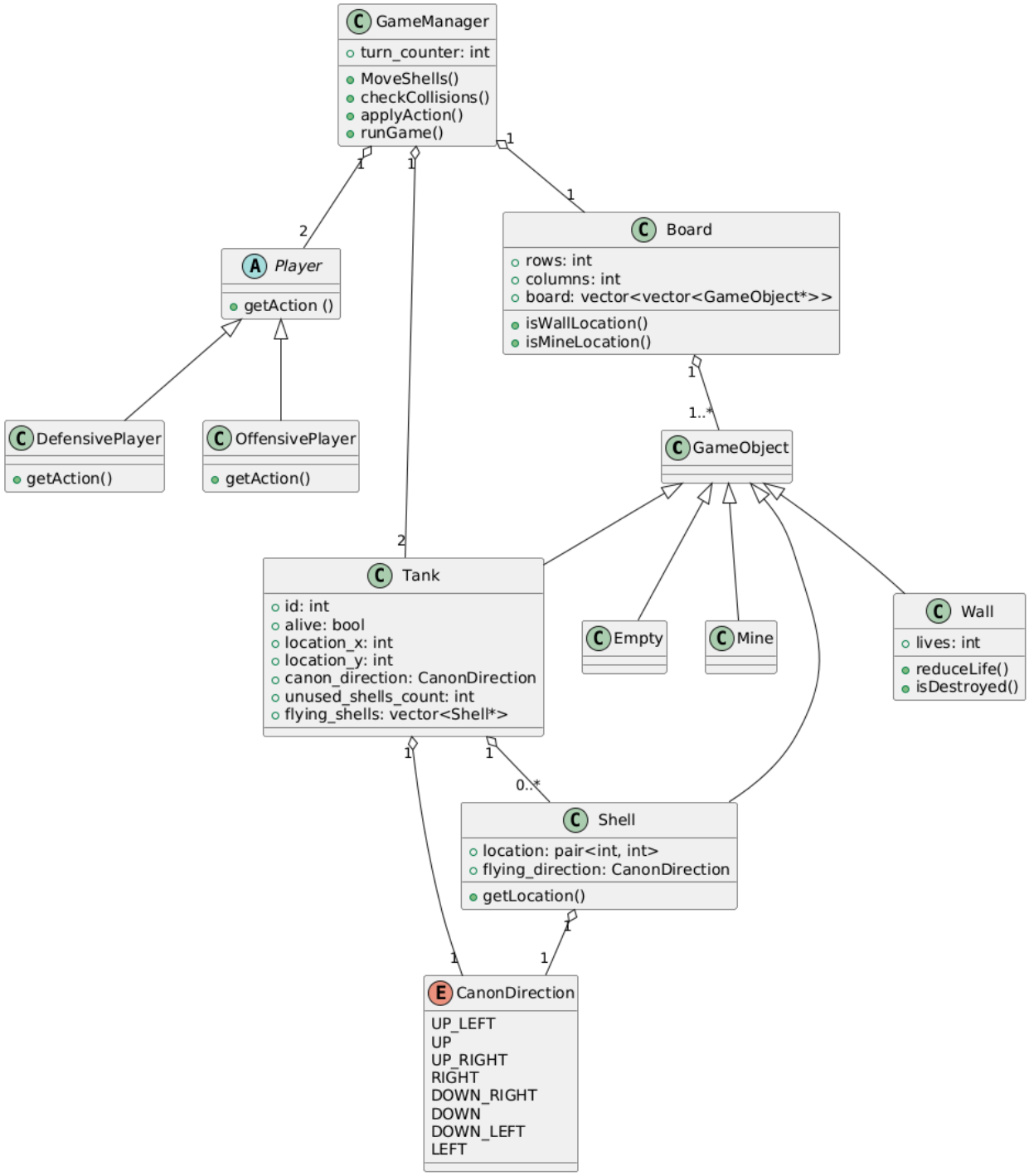
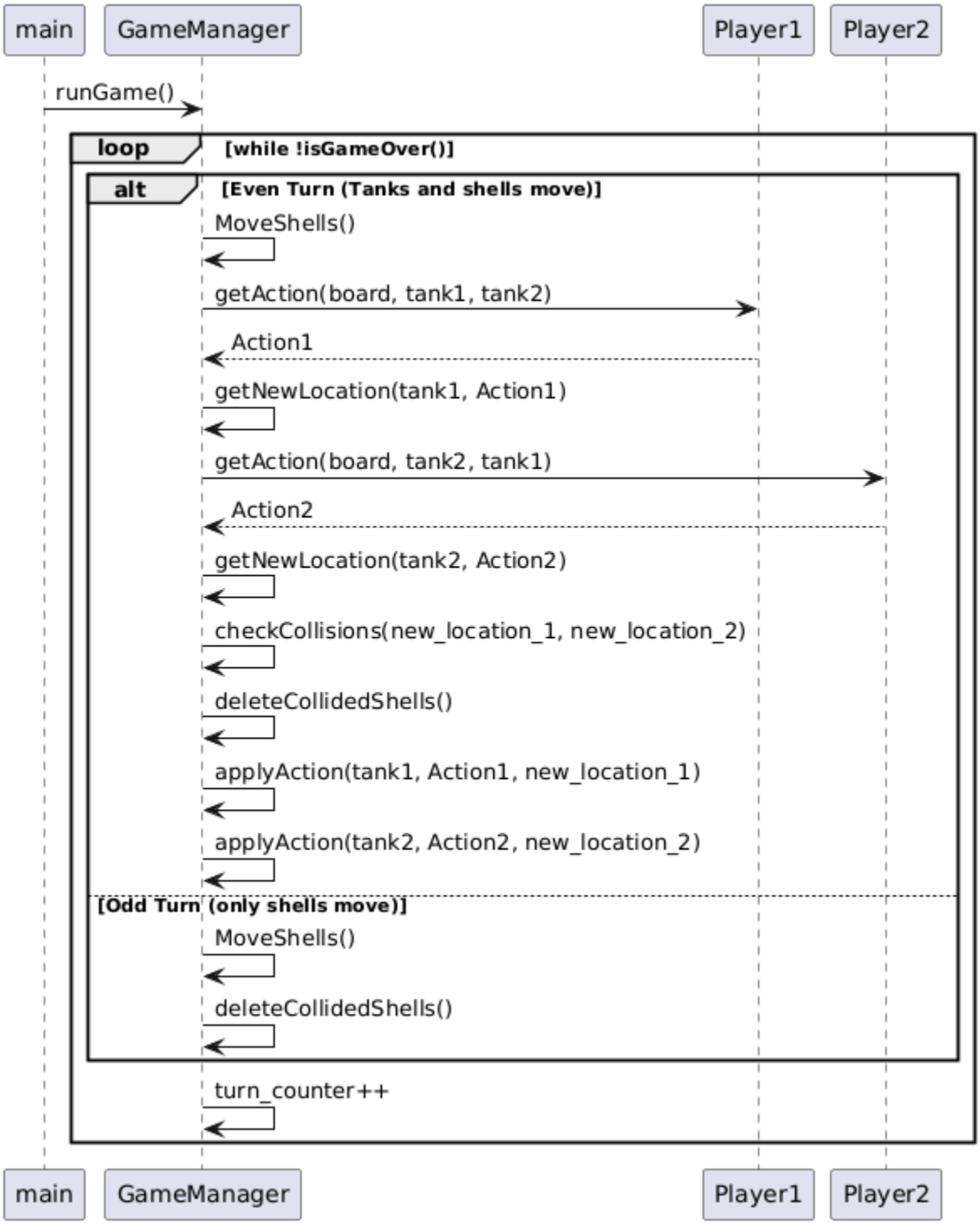
Class Diagram



Sequence Diagram



Design Considerations

Classes and Objects

Every object described in the game instructions received its own class in the code so that it would be an independent entity with appropriate properties and functions.
A game is defined by:

- A board, which is a matrix of game objects
- Two tanks, each with an identifier according to the player
- Two players

All of these are held by the game manager as class variables, and it is responsible for their operation and for the flow of the game.
Additionally, due to this structure and the given instructions in the assignment, the game manager is responsible for checking the validity of all moves and all edge cases that may occur in the game.
Its behavior was designed to support reasonable scenarios, as discussed in the forum.
We designed the players as independent entities from the game, under the assumption that player code may be added in the future without knowing anything about the internal implementation of the game.
Therefore, we designed the connection between the game and the players to include only what is necessary –
the player receives from the game manager the board and the tank positions (as constant references) and only returns an action it wants to perform.
Communication between the player and the game manager is done via a shared Enum called Action, which represents a wanted action from the player.
The player is intentionally unable to modify the game state so it cannot carry out malicious intentions.
Additionally, the player has internal logic that it also does not expose, and it only presents the game manager with a final decision in the form of an action request.

Game Flow

In order to allow different speeds for shells and tanks, we chose to implement the game loop according to the advice given in the forum: doubling the turns and dividing them into even and odd turns.

In this way, during even turns, both tanks and shells move, and in odd turns, only the shells move.

In each type of iteration (even or odd) the possible objects collisions are checked and handled appropriately.

Thus, each game turn consists of two iterations of the loop, resulting in the desired behavior.

Testing approach

We tested the game in an "end-to-end" approach, meaning each test is a game, with two players and a board. The test is running the game and checking (manually) that the output is what should happen according to the rules.

This method allowed us to test multiple aspects and features of the game and check functionality all together. Since we didn't design the project for unit testing, we struggled to check each functionality alone so end-to-end tests were very useful.

The tests start with very simple cases and evolve into more complex games, to do so we created some players that are not reasonable.

For example "RotatingPlayer" only stays in place and rotating, "SimplePlayer" only moves forward and "SecondPlayer" only shoots, they are located in tests/players directory.

We decided to build the game part by part and run tests after each part, as soon as possible.

We started by creating a game with no shell at all and only forward movement,

After that we immediately checked the movement of tanks, collision of them with all objects and the simple game flow.

After that we handled backward movement and rotating.

Inputs 1,2,3 allowed us to ensure that this very basic functionality works fine and continue safely to handle shells.

Then we added shooting and shell movement.
When we finished that we created Inputs 4,5,6 because we didn't have the complex players yet but we needed to make sure what we wrote works.
These tests allowed us to continue working on the players,
After creating the defensive and offensive players we used Inputs 7,8,9,10,11,12 to ensure they work properly and to check the game completely with all features.
In the file test.cpp there is a code that creates a game from each input and runs it.

Note - The tests are in the tests directory, There are also input and output directories in the main directory with 3 selected boards, they are named “a”, “b” and “c” as requested in the submission guidelines, these boards are the same as tests inputs 9, 11 and 12.

Explanation for each test input:
Input 2 - both players are moving forward towards each other, should collide and the game finishes with a tie.
Input 3 - both players are moving forward, 1 should get to the border and jump to the other side, 2 will go over the mine and explode, 1 wins.
Input 4 - both players are shooting at each other, shells should always collide, players continue to shoot until they run out of shells and the game finishes with a tie. make sure players wait 4 turns before shooting again.
Input 5 - player 1 shoot and player 2 does nothing, shell should hit player 2, player 1 wins.
Input 6 - player 1 moves forward and hits the wall, player 2 shoots, wall should be destroyed after 2 hits and player 1 should move until shell hits him.
Input 7 - player 2 is offensive and player 1 is just rotating, player 2 should move until it can shoot from diagonal and win .
Input 9 - player 2 is offensive and player 1 is defensive. tank 2 should shoot towards 1, the shell will wrap around and hit him. Player 2 wins.
Input 10 - player 2 is offensive and player 1 is defensive. Long game until player 2 manages to shoot on player 1 and win.

Input 11 - player 2 is offensive and player 1 is defensive, but player 1 is getting to a good position and able to shoot before player 2 and win. Also shows shell wrap around the board.
Input 12 - same board as input 4, but player 1 is defensive and player 2 is offensive. Both tanks shoot shells, and they collide. The defensive player chose to rotate before he knows the shells will collide hoping to escape, and then when he can move backwards he does that because his default behaviour is trying to move. The offensive player stays in place because he knows he is in position to shoot and after waiting enough time he shoots again at player 1, who cannot escape and also couldn't shoot back in time.

Games that are going to run **forever** (should run alone and stopped manually)
Input 1 - both players stays in place (just rotating), game should run forever
Input 8 - player1 is defensive player2 is offensive. player2 fails to shoot at 1 and the game runs forever.